MCMP117 · WEEK 14

Fundamentals of Operating Systems

Week 14 · Introduction to Shell Scripting

Type it once, save it, run it a thousand times.

Electronical Communication Technology Programme · Vocational School · Istanbul Okan University

LAST WEEK

Week 13 in Four Ideas

Processes

Every command runs as a process with a PID and a parent (PPID). ps, pstree and top show them; STAT shows R, S, T, Z.

Job control

command & runs in the background; Ctrl+Z stops a job; jobs, fg and bg move jobs between foreground and background.

Signals

kill sends a signal: SIGTERM (15) asks politely, SIGKILL (9) cannot be refused, Ctrl+C sends SIGINT (2).

Priority

nice and renice set the nice value (−20 … 19); only root may lower it (raise the priority).

Today: a script is just another program. When you run it, the shell creates a new process for it — everything from Week 13 applies.

Today's Agenda

1. **Your first script** Why automate; the shebang; chmod +x; ./script.sh vs. bash script.sh.
2. **Variables & input** name=value, quoting, $( ), $(( )), $1 $# $@, read.
3. **Decisions** Exit status $?, test and [ ], if / elif / else, debugging with bash -x.
4. **Loops & functions** for (lists, {1..5}, files), while, functions.
5. **Two real scripts** backup.sh (timestamped tar.gz) and report.sh (log and grade summary).
6. **Lab** Common mistakes, then you write 3–4 graded scripts.

PART 1

Your First Script

A shell script is a text file full of commands.

AUTOMATION

Why Write Scripts?

A shell script is a plain text file that contains shell commands. The shell reads it and runs the commands, one after another.

Typing commands by hand

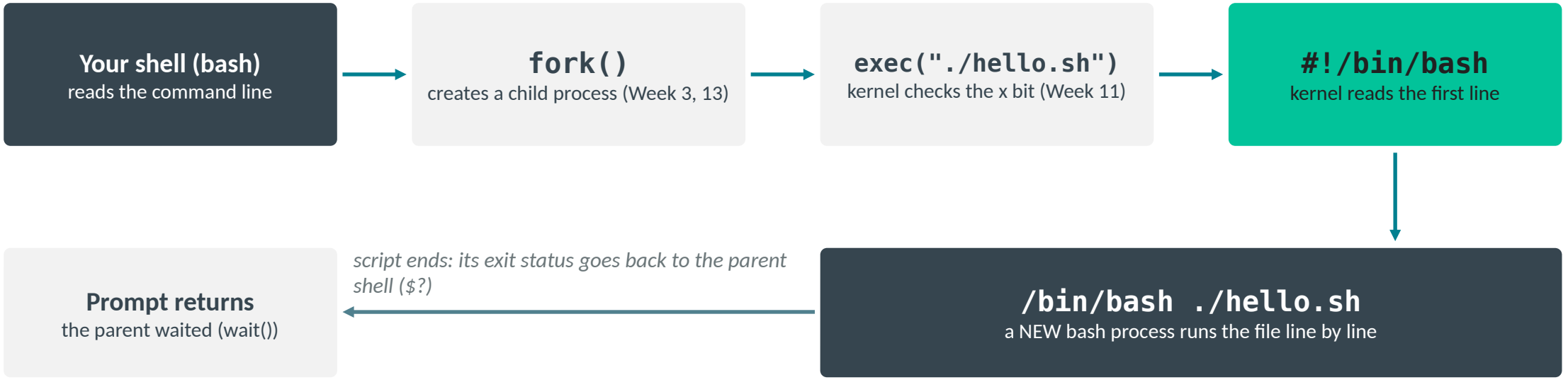
- Every morning: 6 commands to back up and check the logs.
- Easy to forget a step or mistype a path.
- Hard to explain to a colleague exactly what you did.
- Cannot run at 03:00 while you sleep.

Putting them in a script

- One command: ./backup.sh — same steps every time.
- Fix a mistake once; it stays fixed.
- The script documents the procedure (comments!).
- Can be started automatically by a scheduler (cron).

Rule of thumb: if you type the same sequence of commands a third time, write a script. System administrators, DevOps and embedded-Linux engineers do this every day.

What Happens When You Type ./hello.sh?



Consequences

- The script runs in a child process: a cd or a variable inside the script does not change your terminal.
- No x permission → exec fails → "Permission denied".

The shebang line #!

- Must be the very first line: #! + absolute path of the interpreter.
- #!/bin/bash for bash; #!/usr/bin/python3 would run Python the same way.

Anatomy of hello.sh

```
#!/bin/bash
# hello.sh -- my first shell script
# Run: bash hello.sh   or: chmod +x hello.sh; ./hello.sh
echo "Hello from a shell script!"
echo "I am running as: $(whoami)"
echo "Current folder:  $(pwd)"
```

1. Shebang: which program runs this file
2. Comments: ignored by bash, read by humans
3. Commands: exactly what you would type

Create it with a text editor

- **nano hello.sh** — type the lines, save with Ctrl+O, Enter, exit with Ctrl+X. (Any editor works: vim, gedit, VS Code.)
- **$(whoami)** is command substitution: bash runs whoami and puts its output into the text (Part 2).
- **Name:** the .sh ending is only a convention for humans; Linux decides by the x bit and the shebang, not by the extension (Week 7: file types).

LIVE DEMO

Running It: bash hello.sh vs. ./hello.sh

```
student@okan-lab:~/lab/scripts$ bash hello.sh
Hello from a shell script!
I am running as: student
Current folder:  /home/student/lab/scripts
student@okan-lab:~/lab/scripts$ ./hello.sh
bash: ./hello.sh: Permission denied
student@okan-lab:~/lab/scripts$ echo $?
126
student@okan-lab:~/lab/scripts$ ls -l hello.sh
-rw-rw-r-- 1 student student 204 Oct 12 17:18 hello.sh
student@okan-lab:~/lab/scripts$ chmod +x hello.sh
student@okan-lab:~/lab/scripts$ ls -l hello.sh
-rwxrwxr-x 1 student student 204 Oct 12 17:18 hello.sh
student@okan-lab:~/lab/scripts$ ./hello.sh
Hello from a shell script!
I am running as: student
Current folder:  /home/student/lab/scripts
student@okan-lab:~/lab/scripts$ hello.sh
bash: hello.sh: command not found
```

What to observe

- **bash hello.sh** works without x: bash only reads the file.
- **./hello.sh** needs the x bit: Permission denied, exit status 126.
- **chmod +x** adds x for user, group and others (Week 11): rw- → rwx.
- **hello.sh** alone: not found, because the current folder is not in $PATH. Hence ./

Four Ways to Run a Script

Command	Needs	Where it runs	Typical use
bash hello.sh	r permission	new bash process	quick test; script without x bit
./hello.sh	r + x, shebang line	new process (interpreter from #!)	normal way to run your own script
hello.sh	r + x, folder in $PATH	new process	installed tools, e.g. in ~/bin
source hello.sh (or . hello.sh)	r permission	the CURRENT shell	files that set variables, e.g. ~/.bashrc

Use ./script.sh in this course. bash -n script.sh checks the syntax without running anything; bash -x script.sh shows every command as it runs (Part 3).

PART 2

Variables, Quoting & Input

Storing values, building text, reading arguments and keyboard input.

Variables: name=value, No Spaces!

```
student@okan-lab:~/lab/scripts$ name="Ayse"
student@okan-lab:~/lab/scripts$ echo $name
Ayse
student@okan-lab:~/lab/scripts$ echo "Hello, $name!"
Hello, Ayse!
student@okan-lab:~/lab/scripts$ name = Ayse
bash: name: command not found
```

name = Ayse is read as: run the command name with the two arguments = and Ayse. Hence "name: command not found". The space is not decoration in bash — it separates words.

The rules

- **Assign:** name=value — no spaces around =.
- **Use:** $name or ${name}; an unset variable is simply empty.
- **Names:** letters, digits, _; not starting with a digit; case-sensitive (name ≠ NAME).
- **Everything is text:** count=5 stores the string "5"; arithmetic needs $(( )).
- **Convention:** UPPERCASE for environment variables (HOME, USER, PATH); lowercase for your own.

Command Substitution $( ) and Arithmetic $(( ))

```
name="Ayse"                      # NO spaces around the = sign
course=MCMP117
week=14
echo "Student: $name, course: $course, week $week"

today=$(date +%Y-%m-%d)          # $( ) = the OUTPUT of a command
lines=$(wc -l < ~/lab/docs/syllabus.txt)
echo "Today is $today; the syllabus has $lines lines"

left=$((15 - week))              # $(( )) = integer arithmetic
echo "Weeks left until the final exam: $left"
echo "7 / 2 = $((7 / 2)) remainder $((7 % 2))   (integers only!)"
```

```
student@okan-lab:~/lab/scripts$ ./variables.sh
Student: Ayse, course: MCMP117, week 14
Today is 2026-10-12; the syllabus has 15 lines
Weeks left until the final exam: 1
7 / 2 = 3 remainder 1   (integers only!)
```

Two expansions

- **$(command)** → replaced by the output of the command (trailing newline removed). Old form: `command`.
- **$((expression))** → integer arithmetic: + − * / % and ( ). Inside, $ before names is optional.
- **Integers only:** 7 / 2 = 3, the remainder 7 % 2 = 1. For decimals use awk (report.sh) or bc.

Double Quotes, Single Quotes, No Quotes

```
name="Ayse"
file="final report.txt"          # a file name that contains a space
# shellcheck disable=SC2016      # (on purpose)
echo 'Single: Hello $name'       # '...' : everything is literal
echo "Double: Hello $name"       # "..." : $var and $( ) expanded
echo "Escaped: it costs \$5"     # \$ : one literal dollar sign
cd ~/lab/docs || exit 1
wc -l "$file"                    # quoted:   ONE argument
# shellcheck disable=SC2086      # (on purpose)
wc -l $file                      # unquoted: TWO arguments
```

```
student@okan-lab:~/lab/scripts$ ./quotes.sh
Single: Hello $name
Double: Hello Ayse
Escaped: it costs $5
2 final report.txt
wc: final: No such file or directory
wc: report.txt: No such file or directory
0 total
```

What quotes do

- **"double":** keeps spaces; $var and $( ) are still expanded. Use them almost always.
- **'single':** everything literal, no expansion at all.
- **\ (backslash):** makes one character literal: \$, \".
- **No quotes:** the value is split at spaces (and * is expanded). "final report.txt" becomes two arguments — two errors.

Positional Parameters: $0, $1 … $9, $#, $@

```
echo "Script name  \$0 = $0"
echo "First arg    \$1 = $1"
echo "Second arg   \$2 = $2"
echo "Arg count    \$# = $#"
echo "All args     \$@ =" "$@"
i=1
for arg in "$@"; do             # "green tea" stays ONE word
    echo "  argument $i: [$arg]"
    i=$((i + 1))
done
```

Variable	Meaning
`$0`	name of the script as it was typed
`$1 … $9`	1st … 9th argument (${10} for the 10th)
`$#`	number of arguments
`"$@"`	all arguments, each kept as a separate word
`$?`	exit status of the last command (Part 3)
`$$`	PID of this shell (Week 13)

```
student@okan-lab:~/lab/scripts$ ./args.sh apple "green tea" 42
Script name  $0 = ./args.sh
First arg    $1 = apple
Second arg   $2 = green tea
Arg count    $# = 3
All args     $@ = apple green tea 42
  argument 1: [apple]
  argument 2: [green tea]
  argument 3: [42]
```

The shell splits the command line into words before the script starts: "green tea" is ONE argument because of the quotes.

Reading Keyboard Input with read

```
read -r -p "What is your name? " name
read -r -p "Which year were you born? " year
age=$(( $(date +%Y) - year ))
echo "Nice to meet you, $name. You are about $age years old."
```

```
student@okan-lab:~/lab/scripts$ ./greet.sh
What is your name? Ayse
Which year were you born? 2006
Nice to meet you, Ayse. You are about 20 years old.
```

read in one line

- **read -r var** waits for one line and stores it in var.
- **-p "text"** shows a prompt first.
- **-r** keeps backslashes as typed (always use it).

Arguments ($1) or read? Arguments are better for automation: ./grade.sh 72 can run from another script or from cron with nobody at the keyboard. read is for interactive tools, e.g. a menu or a game (Lab exercise 14).

CHEAT SHEET

Syntax So Far

You write	Bash does	Example (real output)
#!/bin/bash	first line: choose the interpreter	./hello.sh runs with /bin/bash
name=value	assign (no spaces!)	week=14
"$name" '$name'	expand / literal	Hello Ayse / Hello $name
$(command)	insert the output of a command	$(date +%Y-%m-%d) → 2026-10-12
$((expression))	integer arithmetic	$((3 + 4 * 2)) → 11
$1 $# "$@"	arguments, their count, all of them	./args.sh a b → $# = 2
read -r -p "? " x	read one line from the keyboard	x=Ayse

INTERMISSION

10-Minute Break

Next: decisions, loops, functions — and two real automation scripts.

PART 3

Making Decisions

Every command answers a yes/no question: its exit status.

Every Command Returns an Exit Status: $?

```
student@okan-lab:~/lab/scripts$ ls ../docs > /dev/null
student@okan-lab:~/lab/scripts$ echo $?
0
student@okan-lab:~/lab/scripts$ ls ../nothing
ls: cannot access '../nothing': No such file or directory
student@okan-lab:~/lab/scripts$ echo $?
2
student@okan-lab:~/lab/scripts$ nosuchcommand
bash: nosuchcommand: command not found
student@okan-lab:~/lab/scripts$ echo $?
127
student@okan-lab:~/lab/scripts$ grep -q ERROR ../logs/system.log
student@okan-lab:~/lab/scripts$ echo $?
0
student@okan-lab:~/lab/scripts$ grep -q PANIC ../logs/system.log
student@okan-lab:~/lab/scripts$ echo $?
1
```

$?	Meaning
0	success ("true")
1	general failure / "false"
2	wrong usage (ls, grep: error)
126	found, but not executable
127	command not found

0 means success — the opposite of C, where 0 is false! A program sets it with exit(n) or return n from main (Week 3); a script sets it with exit n. Range 0–255.

test and [ ]: A Command That Compares

```
student@okan-lab:~/lab/scripts$ [ 5 -gt 3 ]
student@okan-lab:~/lab/scripts$ echo $?
0
student@okan-lab:~/lab/scripts$ [ "abc" = "abd" ]
student@okan-lab:~/lab/scripts$ echo $?
1
student@okan-lab:~/lab/scripts$ type [
[ is a shell builtin
```

[is a command, so spaces matter

- [5 -gt 3] is the same as test 5 -gt 3. The last argument must be].
- Every part is a separate word: ["$n" -gt 3]. Without spaces, [$n becomes a command name: "[5: command not found".
- Quote variables inside [ ]: an empty $x would vanish and break the test.

Type	Operators	Example
Numbers	-eq -ne -lt -le -gt -ge	["$n" -ge 50]
Strings	= != -z (empty) -n (not empty)	["$ans" = yes]
Files	-e exists -f file -d directory	[-d ~/lab/backup]
	-r readable -w writable -x exec.	[-x backup.sh]
Logic	! (not) && || between tests	[-f a] && [-f b]

Numbers: -lt, -gt … (letters!). Strings: = and !=. Inside [ ], < and > are redirections (Week 12), not comparisons.

if: Run Commands Only When a Test Succeeds

```
if [ $# -ne 1 ]; then                    # exactly one argument?
    echo "Usage: $0 PATH" >&2            # >&2 : message goes to stderr
    exit 2                               # 2 = wrong usage
fi
path=$1
if [ -f "$path" ]; then
    echo "$path: regular file, $(wc -l < "$path") lines"
elif [ -d "$path" ]; then
    echo "$path: directory"
elif [ -e "$path" ]; then
    echo "$path: exists, but is neither a file nor a directory"
else
    echo "$path: does not exist" >&2
    exit 1                               # 1 = failure
fi
exit 0                                   # 0 = success
```

The shape

- if TEST; then … elif TEST; then … else … fi
- TEST can be any command: [ ], grep -q, mkdir …
- fi (if backwards) ends the block. Indent the body by 4 spaces.

Good habits shown here

- Check $# first; print a Usage line.
- Errors go to stderr: >&2 (Week 12).
- Distinct exit codes: 0 ok, 1 fail, 2 usage.

LIVE DEMO

check.sh: Four Kinds of Path, Three Exit Codes

```
student@okan-lab:~/lab/scripts$ ./check.sh ../docs/syllabus.txt
../docs/syllabus.txt: regular file, 15 lines
student@okan-lab:~/lab/scripts$ ./check.sh ../logs
../logs: directory
student@okan-lab:~/lab/scripts$ ./check.sh /dev/null
/dev/null: exists, but is neither a file nor a directory
student@okan-lab:~/lab/scripts$ ./check.sh nothing.txt
nothing.txt: does not exist
student@okan-lab:~/lab/scripts$ echo $?
1
student@okan-lab:~/lab/scripts$ ./check.sh
Usage: ./check.sh PATH
student@okan-lab:~/lab/scripts$ echo $?
2
```

What to observe

- syllabus.txt: -f is true, so the first branch runs.
- ../logs: -f false, -d true: the elif branch.
- /dev/null: exists (-e), but it is a device file (Week 9: /dev), so the else-if chain reaches the third branch.
- Missing file: exit 1. No argument: Usage and exit 2.

grade.sh: if / elif / else with Numbers

```
score=$1
if [ -z "$score" ]; then               # -z : the string is empty
    echo "Usage: $0 SCORE (0-100)" >&2
    exit 2
fi
if [ "$score" -lt 0 ] || [ "$score" -gt 100 ]; then
    echo "Error: score must be between 0 and 100" >&2
    exit 1
elif [ "$score" -ge 85 ]; then
    letter=A
elif [ "$score" -ge 70 ]; then
    letter=B
elif [ "$score" -ge 50 ]; then
    letter=C
else
    letter=F
fi
echo "Score $score -> grade $letter"
```

```
student@okan-lab:~/lab/scripts$ ./grade.sh 92
Score 92 -> grade A
student@okan-lab:~/lab/scripts$ ./grade.sh 72
Score 72 -> grade B
student@okan-lab:~/lab/scripts$ ./grade.sh 49
Score 49 -> grade F
student@okan-lab:~/lab/scripts$ ./grade.sh 150
Error: score must be between 0 and 100
student@okan-lab:~/lab/scripts$ ./grade.sh
Usage: ./grade.sh SCORE (0-100)
```

Read it like a staircase

- bash tries the tests from top to bottom and runs only the FIRST branch whose test is true.
- `||` = or: the second test runs only if the first is false.

Debugging: bash -n, bash -x and Bad Input

```
student@okan-lab:~/lab/scripts$ bash -x grade.sh 72
+ score=72
+ '[' -z 72 ']'
+ '[' 72 -lt 0 ']'
+ '[' 72 -gt 100 ']'
+ '[' 72 -ge 85 ']'
+ '[' 72 -ge 70 ']'
+ letter=B
+ echo 'Score 72 -> grade B'
Score 72 -> grade B
```

```
student@okan-lab:~/lab/scripts$ ./grade.sh abc
./grade.sh: line 9: [: abc: integer expression expected
./grade.sh: line 9: [: abc: integer expression expected
./grade.sh: line 12: [: abc: integer expression expected
./grade.sh: line 14: [: abc: integer expression expected
./grade.sh: line 16: [: abc: integer expression expected
Score abc -> grade F
```

Three tools

- **bash -n script.sh:** syntax check only; nothing runs.
- **bash -x script.sh:** prints every command (with + and the values already filled in) before running it. Inside a script: set -x … set +x around the part you want to trace.
- **Validate input:** grade.sh abc prints errors yet still says grade F — a bug. Check the input first (Lab exercise ideas).

PART 4

Loops & Functions

Do something for every item, or while a condition holds.

for: Once for Every Item in a List

```
for city in Istanbul Ankara Izmir; do       # 1) over a list of words
    echo "City: $city"
done

for i in {1..5}; do                          # 2) over a range of numbers
    echo -n "$i squared is $((i * i)); "
done
echo                                         # finish the line

for f in ~/lab/docs/*.txt; do                # 3) over files (wildcard)
    echo "$(basename "$f"): $(wc -l < "$f") lines"
done
```

```
student@okan-lab:~/lab/scripts$ ./loops.sh
City: Istanbul
City: Ankara
City: Izmir
1 squared is 1; 2 squared is 4; ...
final report.txt: 2 lines
linux_quotes.txt: 5 lines
notes_01.txt: 1 lines
notes_02.txt: 1 lines
notes_03.txt: 1 lines
syllabus.txt: 15 lines
```

for VAR in LIST; do ... done

- **List of words:** written out, or "$@".
- **{1..5}:** brace expansion → 1 2 3 4 5 (Week 10).
- ***.txt:** the shell expands the wildcard to the matching file names — safe even with spaces, if you write "$f".
- **Also:** $(command) as the list, e.g. for u in $(cut -d, -f3 file).

while: Repeat as Long as a Test Succeeds

```
n=3
while [ "$n" -gt 0 ]; do                    # repeat WHILE the test is true
    echo "Countdown: $n"
    n=$((n - 1))
done
echo "Go!"

sum=0
count=0
while read -r x; do                         # one line of the file per turn
    sum=$((sum + x))
    count=$((count + 1))
done < ~/lab/data/numbers.txt               # the whole loop reads this file
echo "$count numbers, sum = $sum, average = $((sum / count)) (integer)"
```

```
student@okan-lab:~/lab/scripts$ ./while.sh
Countdown: 3
Countdown: 2
Countdown: 1
Go!
20 numbers, sum = 11325, average = 566 (integer)
```

Two classic patterns

- **Counter:** change the variable inside the loop — forget n=$((n - 1)) and the loop never ends (Ctrl+C, Week 13).
- **Read a file:** while read -r line; do … done < file reads one line per turn; read fails at end of file and the loop stops.
- **Check:** 11325 / 20 = 566.25, bash prints 566.

Functions: Name a Block of Commands

```
line() {                                # no arguments
    echo "-----------------------------------"
}
count_word() {                          # $1 = word, $2 = file
    local n                             # local: exists only in here
    n=$(grep -c -- "$1" "$2")
    echo "$1 appears on $n lines of $(basename "$2")"
}
is_even() {                             # the exit status is the answer
    [ $(( $1 % 2 )) -eq 0 ]
}
line
count_word ERROR ~/lab/logs/system.log
count_word WARN  ~/lab/logs/system.log
line
for k in 7 12; do
    if is_even "$k"; then echo "$k is even"; else echo "$k is odd"; fi
done
```

```
student@okan-lab:~/lab/scripts$ ./functions.sh
--------------------------
ERROR appears on 13 lines of system.log
WARN appears on 28 lines of system.log
--------------------------
7 is odd
12 is even
```

How they work

- Define before use: name() { … }
- Call like a command: count_word WARN file. Inside, $1 $2 are the function's own arguments.
- local keeps a variable private. The exit status of the last command is the function's answer.

CHEAT SHEET

Control Structures at a Glance

Structure	Syntax (one line)	Ends with
if	if ["$n" -gt 0]; then …; elif …; then …; else …; fi	fi
for (list)	for f in *.txt; do echo "$f"; done	done
for (range)	for i in {1..5}; do echo "$i"; done	done
while	while ["$n" -gt 0]; do n=$((n - 1)); done	done
read a file	while read -r line; do …; done < file.txt	done
function	name() { local x=$1; echo "$x"; }	}
and / or	mkdir -p dir && echo ok; [-f f] || exit 1	—

PART 5

Two Real Automation Scripts

backup.sh and report.sh in ~/lab/scripts

AUTOMATION 1

backup.sh: A Timestamped Archive of docs/

```
lab="$HOME/lab"
src="docs"                                  # folder to save (inside $lab)
dest="$lab/backup"
stamp=$(date +%Y%m%d_%H%M%S)                # e.g. 20261012_143005
archive="$dest/docs_$stamp.tar.gz"

if [ ! -d "$lab/$src" ]; then
    echo "Error: $lab/$src not found" >&2
    exit 1
fi
mkdir -p "$dest"                            # no error if it exists
if tar -czf "$archive" -C "$lab" "$src"; then
    n=$(tar -tzf "$archive" | grep -vc '/$')    # count files, not folders
    echo "Backup created: $archive"
    echo "Files: $n, size: $(du -h "$archive" | cut -f1)"
else
    echo "Error: tar failed" >&2
    exit 1
fi
```

Line by line

- **date +%Y%m%d_%H%M%S** gives a unique, sortable name.
- **[! -d ...]:** stop early if the source is missing.
- **tar -czf A -C dir docs:** create (c) a gzip-compressed (z) archive file (f); -C = change into ~/lab first, so paths start with docs/.
- **if tar ...:** test tar's own exit status.

LIVE DEMO

backup.sh in Action

```
student@okan-lab:~/lab/scripts$ ./backup.sh
Backup created: /home/student/lab/backup/docs_20261012_085013.tar.gz
Files: 7, size: 4.0K
student@okan-lab:~/lab/scripts$ ls -l ../backup
total 4
-rw-rw-r-- 1 student student 862 Oct 12 08:50 docs_20261012_085013.tar.gz
student@okan-lab:~/lab/scripts$ tar -tzf ../backup/docs_*.tar.gz
docs/
docs/notes_02.txt
docs/syllabus.txt
docs/.settings
docs/notes_01.txt
docs/notes_03.txt
docs/linux_quotes.txt
docs/final report.txt
student@okan-lab:~/lab/scripts$ ./backup.sh
Backup created: /home/student/lab/backup/docs_20261012_085015.tar.gz
Files: 7, size: 4.0K
student@okan-lab:~/lab/scripts$ ls ../backup
docs_20261012_085013.tar.gz  docs_20261012_085015.tar.gz
```

What to observe

- Each run makes a new file: the timestamp is part of the name, nothing is overwritten.
- Files: 7 — six visible documents plus the hidden .settings (tar includes hidden files).
- "final report.txt" is archived correctly: the script quotes every variable.
- Names sort by time because the date is written year-first.

report.sh: Summarising a Log File and a CSV

```
log="$HOME/lab/logs/access.log"
csv="$HOME/lab/data/students.csv"
for f in "$log" "$csv"; do
    if [ ! -r "$f" ]; then echo "Error: cannot read $f" >&2; exit 1; fi
done
echo "===== Lab report, $(date '+%Y-%m-%d %H:%M') ====="

echo "--- Web server: $(basename "$log") ---"
total=$(wc -l < "$log")
errors=$(awk '$9 >= 400' "$log" | wc -l)    # field 9 = status code
echo "Requests: $total, errors (4xx/5xx): $errors ($((100 * errors / total))%)"
echo "Top 3 pages:"
awk '{print $7}' "$log" | sort | uniq -c | sort -rn | head -3

echo "--- Students: $(basename "$csv") ---"
echo "Students: $(tail -n +2 "$csv" | wc -l)"
echo "Per department:"
tail -n +2 "$csv" | cut -d, -f3 | sort | uniq -c
awk -F, 'NR > 1 { m += $4; f += $5; n++ }
         END { printf "Average midterm %.1f, average final %.1f\n", m/n, f/n }' "$csv"
passed=$(awk -F, 'NR > 1 && 0.4*$4 + 0.6*$5 >= 50' "$csv" | wc -l)
echo "Passed (0.4 x midterm + 0.6 x final >= 50): $passed"
```

LIVE DEMO

report.sh: Real Output, Checked

```
student@okan-lab:~/lab/scripts$ ./report.sh
===== Lab report, 2026-10-12 08:50 =====
--- Web server: access.log ---
Requests: 300, errors (4xx/5xx): 98 (32%)
Top 3 pages:
     41 /courses/mcmp117
     37 /about.html
     33 /index.html
--- Students: students.csv ---
Students: 40
Per department:
     10 CS
     17 ECT
      2 EE
     11 ME
Average midterm 67.0, average final 65.6
Passed (0.4 x midterm + 0.6 x final >= 50): 35
```

Numbers verified (Python)

- **Errors:** 98 of 300 requests have status ≥ 400; 100·98/300 = 32.67, bash integer arithmetic prints 32.
- **Ties:** /index.html and /images/logo.png both have 33 hits; sort decides the order.
- **Departments:** 17 + 11 + 10 + 2 = 40.
- **Averages:** midterm 66.95 → printed 67.0; final 65.6.
- **Passed:** 35 of 40 have 0.4m + 0.6f ≥ 50.

Mistakes You Will Make (Everybody Does)

Spaces around =

```
name = "Ayse"
echo "Hello $name"
$ ./bad_space.sh
./bad_space.sh: line 2: name: command
not found
Hello
```

Fix: name="Ayse"

No spaces inside [ ]

```
n=5
if [$n -gt 3]; then echo "big"; fi
$ ./bad_test.sh
./bad_test.sh: line 3: [5: command
not found
```

Fix: if ["$n" -gt 3]; then

Missing quotes

```
answer=""
if [ $answer = yes ]; then echo "OK";
fi
$ ./bad_quotes.sh
./bad_quotes.sh: line 3: [: =: unary
operator expected
```

Fix: ["$answer" = yes]

Read the error message: it names the script, the line number and the word bash could not understand.

Windows Line Endings, Missing fi, Missing chmod

```
student@okan-lab:~/lab/scripts$ ./win.sh
bash: ./win.sh: cannot execute: required file not found
student@okan-lab:~/lab/scripts$ bash win.sh
win.sh: line 2: cd: $'/home/student/lab\r': No such file or directory
Hello from Windows
student@okan-lab:~/lab/scripts$ cat -A win.sh
#!/bin/bash^M$
cd ~/lab^M$
echo "Hello from Windows"^M$
student@okan-lab:~/lab/scripts$ sed -i 's/\r$//' win.sh
student@okan-lab:~/lab/scripts$ ./win.sh
Hello from Windows
```

```
student@okan-lab:~/lab/scripts$ bash -n bad_if.sh
bad_if.sh: line 4: syntax error: unexpected end of file
```

CRLF: the invisible bug

- Windows editors end lines with \r\n; Linux expects \n. The \r (shown as ^M) sticks to the words: the kernel looks for an interpreter called "/bin/bash\r".
- Newer bash: "cannot execute: required file not found"; older bash (e.g. Ubuntu 22.04): "/bin/bash^M: bad interpreter".
- Fix: sed -i 's/\r$//' file, or dos2unix file; in VS Code set LF.
- Missing chmod +x → Permission denied (slide 8).

LAB · GRADED

In-Class Lab: Write Four Scripts (35 Minutes)

1	`power.sh N`	Print N, N^2 and N^3. Without an argument: a Usage line and exit 2.	*Ex. 2*
2	`what.sh PATH…`	For every argument say file (with line count), directory or missing; exit 1 if any is missing.	*Ex. 6*
3	`levels.sh`	Count the INFO, WARN and ERROR lines of ~/lab/logs/system.log with a for loop and grep -c.	*Ex. 11*
4	`backup_keep.sh`	Challenge: extend backup.sh so that only the 3 newest archives are kept.	*Ex. 13*

Grading: runs without errors (40 %), correct output (30 %), handles bad input (15 %), comments + quoting + shebang (15 %). Test each script with bash -n first. Submit the four .sh files as announced on the course Telegram channel.

QUICK QUIZ

Check Your Understanding

1. Why does ./hello.sh print "Permission denied" while bash hello.sh works? Which exit status do you get?

2. x=7; y=2. What do echo "$((x / y))", echo '$x' and echo "xy" print?

3. Write a test that is true when the file report.txt exists AND the variable n is greater than 10.

4. ./args.sh one "two three" four — what are $# and $2?

Week 14 — Key Takeaways

- **Script:** a text file of commands; #!/bin/bash on line 1, chmod +x, run with ./script.sh. It runs in a child process (fork + exec).
- **Variables:** name=value without spaces; "$name" in double quotes; $( ) inserts command output; $(( )) does integer arithmetic.
- **Input:** arguments $1 … $9, $#, "$@"; keyboard with read -r.
- **Decisions:** every command has an exit status (0 = success); [ ] tests numbers (-lt), strings (=, -z) and files (-f, -d, -e); if / elif / else / fi.
- **Loops & functions:** for over lists, {1..5} and *.txt; while with a counter or reading a file; name() { … }.
- **Debugging:** bash -n (syntax), bash -x (trace); read the error line number; beware CRLF line endings.

NEXT WEEK

Final Exam Review (Week 15)

Week 15 will cover:

- Whole course: theory Weeks 1–7 and Linux Weeks 9–14
- Practice problems: scheduling, memory, page replacement
- "Write the command / pipeline / script" questions
- A sample final paper with an answer key
- Exam date and room: check the course Telegram channel

QUESTIONS?

Discussion

If you have questions after class, ask them on the course Telegram channel — everyone benefits from the answer.

Asst. Prof. Dr. Hasan Oğuz